XML job importer exercise

Build a small Node.js/TypeScript command-line tool that keeps a saved job list up to date from XML files. It must handle new and changed jobs without losing good data when the input has errors.

Spend no more than 2 hours on the build. Stop after 2 hours and note any shortcuts or unfinished work. You may use AI tools, including Codex or Claude.

What to build

- Read one local XML file each time you run the tool. No API or credentials are needed.
- Save the job list in a local JSON file. Start with an empty list if that file does not exist.
- For each job, store its source ID, title, location, description, apply URL and status. Status must be “open” or “closed”, based on the rules below; it is not supplied in the XML.

Input and field rules

Use the supplied development feed. Each file uses this structure: a <jobs> root containing <job> elements. Each job may contain <id>, <title>, <location>, <description> and <applyUrl>. Ignore other tags.

- Keep IDs as strings. Remove whitespace from the beginning and end of id, title and applyUrl. Each must contain a value after this step.
- applyUrl must be a full HTTP or HTTPS URL, such as https://example.com/jobs/123.
- If location or description is missing, store an empty string. Otherwise, preserve its text without further cleanup or normalization.
- Description HTML may be escaped XML text or inside CDATA. Handle both forms.

When to reject the whole file

Report the problem and leave the existing JSON file untouched if:

- The input file cannot be read.
- The XML cannot be parsed.
- The root element is not <jobs>.
- Any job is missing an ID, or its ID contains only whitespace.

These failures stop the entire import. Do not add, update or close any jobs from that file.

When to skip one ID

If every job has an ID and the file passes the checks above, handle errors for individual IDs as follows:

- If a row has an ID but fails a field rule, reject that ID for this run. This applies even if another row with the same ID is valid.
- If the same ID appears in several valid rows with identical job data, import it only once. Compare the five source fields using the trimming and missing-field rules above.
- If valid rows for the same ID contain different job data, reject that ID for this run.

For a rejected ID, do not add or update its record. If it was saved previously, keep its existing data and status. Do not close it: its ID is present in the new file. Continue processing other valid IDs.

Report invalid rows and conflicting IDs, with the reason for each rejection.

How to update the saved jobs

Each XML file represents the complete current list of jobs, not just the changes since the last run. After applying the checks above:

1. Add jobs with new accepted IDs. Set their status to “open”.
2. Update changed data for existing accepted IDs. Set them to “open”, including jobs that were previously closed.
3. Mark previously saved jobs as “closed” if their IDs are completely missing from the new file. Keep their records.
4. A valid empty <jobs/> file closes all saved jobs. It does not delete them.
5. Running the same file again immediately must leave the saved job list unchanged and must not create duplicates.

Example: Job 123 is saved as open. If the next file contains job 123 with an invalid apply URL, keep its old data and leave it open. If job 123 is missing from the next file, close it. If it later returns as a valid job, reopen it.

What to print after each run

Print counts of jobs added, updated, unchanged and newly closed. Include reopened jobs in the updated count. Also report invalid rows and conflicting IDs, with reasons.

What to submit

- A Node.js/TypeScript app that runs with one command, plus a shared GitHub link for code review.
- A short README with setup and run instructions, decisions and limitations, any unfinished work, how you used and verified AI output, and a few verification examples.

A full test suite is not required.

What we will check in the demo

Prepare a short walkthrough using the development feed. We will also provide two files you have not seen before and ask you to:

1. Start with a fresh saved job list and import the first file.
2. Import the second file, which contains added, changed and removed jobs. Show the updates, including missing jobs becoming closed.
3. Immediately import the second file again. Show that nothing changes.
4. Import the first file again. Show that a closed job reopens when it returns.

We may also introduce invalid rows, duplicate IDs or malformed XML and discuss how your tool handles them.

Not required

Do not build a UI, API, mock server, database, deployment setup or scheduler. You do not need to physically delete jobs, support other feed formats, or normalize locations or descriptions.